COGS 181 - Homework 5

due Thursday, March 5 at 11:59PM

Instructions: Answer the questions below, attach your code, and insert figures to create a PDF file; submit your file via Gradescope. You may look up the information on the Internet, but you must write the final homework solutions by yourself.

Late Policy: 5% of the total points will be deducted on the first day past due. Every 10% of the total points will be deducted for every extra day past due.

System Setup: You can install Anaconda to setup the Jupyter Notebook environment. Most packages have been already installed in Anaconda. If some package is not installed, you can use `pip` to install the missing package, that is, just type `pip install PACKAGE_NAME` in the terminal.

Problem 1. (12 points) Conceptual Questions

1.1 (3 points) Given an n-character sequence, we want to predict which character would be the n + 1 th character in the sequence. For example, our input is "neura" (which is a 5-character sequence) and we have to predict what would be the 6th character.

Which neural network architecture would be **most** suitable to complete this task?

A. Fully-Connected Feed-Forward Network

B. Convolutional Neural Network

C. Recurrent Neural Network

D. Hopfield Network

Solution: C. Recurrent Neural Network

1.2 (3 points) What is the technical difference between the vanilla backpropagation algorithm and the backpropagation through time (BPTT) algorithm? Choose **one**.

A. Unlike backprop, in BPTT we sum up gradients for corresponding weight **across all** time steps.

B. Unlike backprop, in BPTT we multiply gradients for corresponding weight **across all** time steps.

C. Unlike backprop, in BPTT we take the maximum gradient for corresponding weight **across all** time steps.

Solution: A. Unlike backprop, in BPTT we sum up gradients for corresponding weight **across all** time steps.

1.3 (3 points) What is the main idea behind self-attention in Transformers?

A. To allow the network to attend to different parts of the input sequence with varying degrees of importance.

B. To train the network to classify the input sequence into different categories based on their features.

C. To reduce the computational complexity of the network by learning a compact representation of the input sequence.

D. To enable the network to learn multiple representations of the same input sequence simultaneously.

Solution: A. To allow the network to attend to different parts of the input sequence with varying degrees of importance.

1.4 (3 points) Which of the following statements is true about the differences between Recurrent Neural Networks (RNNs) and Transformers?

A. RNNs can handle variable-length inputs more efficiently than Transformers.

B. Transformers have a better ability to capture long-term dependencies than RNNs due to their self-attention mechanism.

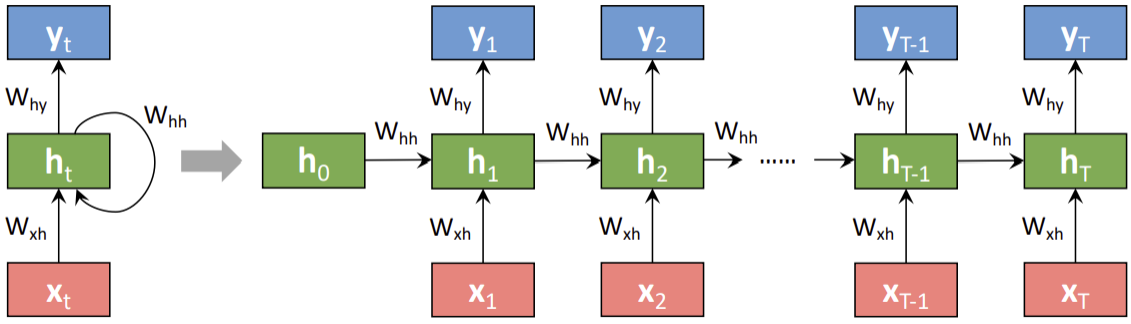
C. Both RNNs and Transformers can handle sequential data, but RNNs are better suited for tasks that require online processing.

D. Transformers are more computationally efficient than RNNs and can handle longer input sequences.

Solution: B. Transformers have a better ability to capture long-term dependencies than RNNs due to their self-attention mechanism.

Problem 2. (20 points) Recurrent Neural Network

In this section, we will use a recurrent neural network (RNN) to process input sequence $\mathbf{x}_t = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{T-1}, \mathbf{x}_T\}$ where $\mathbf{x}_t \in \mathbb{R}^m$ and predict output sequence $\mathbf{y}_t = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_{T-1}, \mathbf{y}_T\}$ where $\mathbf{y}_t \in \mathbb{R}^n$. The RNN's structure is shown below:



The left part of the figure is a common illustration of RNN, which includes three parameters $W_{xh} \in \mathbb{R}^{m \times l}$, $W_{hh} \in \mathbb{R}^{l \times l}$, $W_{hy} \in \mathbb{R}^{l \times n}$ and a hidden state $\mathbf{h}_t \in \mathbb{R}^l$. It includes two steps:

1. Compute hidden state: $\mathbf{h}_t = \text{ReLU}(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1})$.
2. Predict the output: $\mathbf{y}_t = W_{hy}\mathbf{h}_t$.

The right part of the figure shows the "unrolled" version of the RNN, which is the real computing path of the network:

1. In the initial stage, you have the initial hidden state $\mathbf{h}_0$.
2. Then, after feeding the first input $\mathbf{x}_1$ of the sequence, RNN will compute the current hidden state $\mathbf{h}_1 = \text{ReLU}(W_{xh}\mathbf{x}_1 + W_{hh}\mathbf{h}_0)$ where $\text{ReLU}(z) = \max(0, z)$.
3. Afterwards, the RNN will compute the output $\mathbf{y}_1 = W_{hy}\mathbf{h}_1$ and enter next round.
4. In the next round, the RNN will generate hidden state $\mathbf{h}_2 = \text{ReLU}(W_{xh}\mathbf{x}_2 + W_{hh}\mathbf{h}_1)$ and the output $\mathbf{y}_2 = W_{hy}\mathbf{h}_2$. Eventually, the RNN will compute for T rounds and predict the whole output sequence $\{\mathbf{y}_t\} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_{T-1}, \mathbf{y}_T\}$.

Assume $m = 1$, $n = 1$, $l = 2$. We are given the initial hidden state $\mathbf{h}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, the parameters $W_{hh} = \begin{bmatrix} 1 & -1 \\ 2 & -3 \end{bmatrix}$, $W_{xh} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ and $W_{hy} = \begin{bmatrix} -2 & 3 \end{bmatrix}$, and an input sequence $\{\mathbf{x}_1 = 3, \mathbf{x}_2 = 5\}$.

Please calculate the values of the output $\mathbf{y}_1$ and $\mathbf{y}_2$ using the unrolled version of the RNN.

Solution:

$$\mathbf{h}_1 = \text{ReLU}(W_{xh}\mathbf{x}_1 + W_{hh}\mathbf{h}_0) = \text{ReLU}\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}(3) + \begin{bmatrix} 1 & -1 \\ 2 & -3 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

$$\mathbf{y}_1 = W_{hy}\mathbf{h}_1 = \begin{bmatrix} -2 & 3 \end{bmatrix}\begin{bmatrix} 3 \\ 0 \end{bmatrix} = -2 \cdot 3 + 3 \cdot 0 = -6$$

$$\mathbf{h}_2 = \text{ReLU}(W_{xh}\mathbf{x}_2 + W_{hh}\mathbf{h}_1) = \text{ReLU}\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}(5) + \begin{bmatrix} 1 & -1 \\ 2 & -3 \end{bmatrix}\begin{bmatrix} 3 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 8 \\ 1 \end{bmatrix}$$

$$\mathbf{y}_2 = W_{hy}\mathbf{h}_2 = \begin{bmatrix} -2 & 3 \end{bmatrix}\begin{bmatrix} 8 \\ 1 \end{bmatrix} = -2 \cdot 8 + 3 \cdot 1 = -16 + 3 = -13$$

Problem 3. (22 points) Hopfield Network

In this homework problem, we will write a program to "train" and simulate a Hopfield network that memorizes 1-D vector patterns.

Training (Constructing Weights)

Instead of "training" a Hopfield network in the sense that you're used to, the weight matrix in Hopfield networks can be calculated directly using a formula provided in the original paper. Suppose we have N nodes and M states and (here $M = 2$) is the number of states to calculate W and can be retrieved by using W. For each possible state V^s, $s = 1, \ldots, M$, the value of elements in V^s is either -1 or +1. We can compute the weight matrix using:

$$W_{ij} = \frac{1}{N} \sum_{s=1}^{M} V_i^s V_j^s, \quad \text{if } i \neq j \tag{1}$$

And the diagonal values of the weight matrix are all zero, i.e. $w_{ii} = 0$.

Probing Pattern

Given an unknown pattern V_{new}, you will initialize the Hopfield network output nodes at time t, $U_i(t)$ as follows:

$$U_i(0) = V_i^{new}, \quad i = 1, \ldots, n \tag{2}$$

Dynamic Evolution

The inference procedure is very similar to what you have done in perceptron. Iterate until convergence:

$$U_i(t+1) = f_h \left(\sum_{j=1, j \neq i}^{N} W_{ji} U_j(t) \right) \tag{3}$$

$f_h(x)$ is a hard-limiting non-linearity function, where $f_h(x) = +1$ when $x \geq 0$ and $f_h(x) = -1$ when $x < 0$.

In this step, you need to determine an order for updating the nodes **asynchronously which means all units should be updated in sequential at time** t. Typically we use a random visiting sequence (e.g. for a 4-node network, an order can be 2,3,4,1,2,3,4,1,2,3,4,1...).

The stopping criterion is when the outputs from the node remain unchanged. This means that if you go through N consecutive neurons without changing any of their values, then you're at a fixed point so you can stop.

Hint:

- At time t, U should be updated N times.
- LastU stores the value of U before update and is only used to compare with U after N-times update to stop the evolution.

Your Tasks

Write code to build a 5-node Hopfield Network which can converge to the following two states respectively following below procedure:

- (-1, +1, +1, -1, +1)
- (+1, -1, +1, -1, +1)

Please modularize your code to explicitly perform the three steps (Training, Probing Pattern, Dynamic Evolution). For the Probing Pattern step, please use the initial input state vector $V^{new} =$

- (+1, +1, +1, +1, +1)

For the Dynamic Evolution step, please try two different node visiting orders respectively starting from V^{new}:

- $(3, 1, 5, 2, 4, 3, 1, 5, 2, 4...)$
- $(2, 4, 3, 5, 1, 2, 4, 3, 5, 1...)$

Download the skeleton code `hw5-hopfield.ipynb` from the course website. Complete the skeleton code. In order to get full marks, your report should include:

- Attach the code,
- Log the node updating history at each iteration (print out each $U_i(t+1)$ you get during evolution).
- Show the matching between the retrieved pattern (the final fixed-point state your network converges to) and the evolving sequence.

Solution: Code is attached at the end along with the logs.

Starting from $V^{new} = (+1, +1, +1, +1, +1)$

- Order $(3, 1, 5, 2, 4, \ldots)$: converges to $(-1, +1, +1, -1, +1)$.
- Order $(2, 4, 3, 5, 1, \ldots)$: converges to $(+1, -1, +1, -1, +1)$.

Both fixed points match the two stored patterns.

Problem 4. (21 points) Hopfield Network for TSP

In this problem, we will use a Hopfield Network to solve a small instance of the Traveling Salesman Problem (TSP). The goal of TSP is to find a shortest possible tour that visits each city exactly once and returns to the starting city.

Problem Setup

We will solve a TSP that visits 6 cities: San Diego, Los Angeles, Boston, New York, Chicago, Seattle. The reference latitude/longitude coordinates for the cities are given below:

- `San Diego (-117.1611, 32.7157)`
- `Los Angeles (-118.2437, 34.0522)`
- `Chicago ( -87.6298, 41.8781)`
- `New York ( -74.0060, 40.7128)`
- `Boston ( -71.0589, 42.3601)`
- `Seattle (-122.3321, 47.6062)`

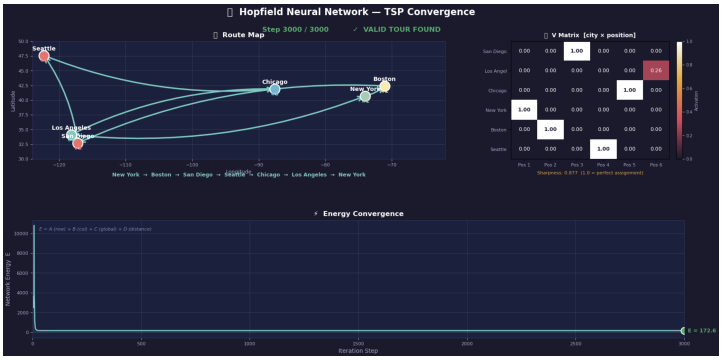
What to Implement

1. **Distance Matrix** Construct a distance matrix $D \in \mathbb{R}^{N \times N}$ from the city coordinates (e.g., Euclidean distance on latitude/longitude). You may normalize distances (recommended) to stabilize optimization.
2. **Hopfield Formulation for TSP** Implement the Hopfield network with an iterative update rule (continuous or discrete) that drives the neuron activations toward a matrix $V \in [0,1]^{N \times N}$, where $V_{i,j}$ encodes assigning city i to tour position j.
3. **Decoding a Tour** Decode the tour from V after training. A valid tour must contain each city exactly once.
4. **Visualization (Required)** Your submission must show:
   - **Route Map**: the evolving/final tour on a 2D map (lon/lat scatter + connections)
   - **V Matrix Heatmap**: activations as a heatmap [city × position], including all 6 cities.
   - **Energy Curve**: energy vs. iteration steps (should generally decrease)

No skeleton code is provided for this task. You may refer to the demo notebook on the course website. You may use it as a starting point, but note that it has a bug (the V matrix for the Los Angeles row is all 0). **You are encouraged to use coding assistants (e.g., Claude Sonnet, Cursor, Copilot) to support vibe coding and debugging for this problem.**

Your report should include the code, and the visualization results mentioned above

Solution: Code is attached at the end. The decoded tour is $(3, 4, 0, 5, 2, 1)$.



Problem 5. (25 points) RNN for Periodic Function Modeling

In this section, you will train an RNN for periodic function modeling. Assume we have a periodic function $f(t) : \mathbb{R} \to \mathbb{R}$ to model:

$$f(t) = \sin(at + b)$$

We focus on modeling the sequence $\{f(t)\} = \{f(1), f(2)..., f(T-1), f(T)\}$. Specifically, we aim to train a network to predict the next value of the function $f(t+1)$ given the current value of the function $f(t)$ and previous hidden state $\mathbf{h}_{t-1}$. Thus, after training, if we have the initial value of function $f(1)$ and initial hidden state $\mathbf{h}_0$, we can iteratively generate the whole sequence $\{f(t)\}$. The following steps are used for sequence generation:

1. We use the current value of function $f(t)$ as input $\mathbf{x}_t \in \mathbb{R}^1$.
2. Based on previous hidden state $\mathbf{h}_{t-1}$ and input $\mathbf{x}_t$, we can calculate the current hidden state $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1})$ and predict the output: $\mathbf{y}_t = W_{hy}\mathbf{h}_t \in \mathbb{R}^1$.
3. We will use the current output $\mathbf{y}_t$ as next value of function $f(t+1)$ and perform the above steps again.

In the evaluation stage, if we have initial hidden state $\mathbf{h}_0$, initial value of function $f(1)$ and trained weight W_{xh}, W_{hh}, W_{hy}, we can utilize the above steps to generate the whole sequence $\{f(1), f(2), ..., f(T)\}$. Besides, if we have a partial sequence $\{f(1), f(2), ..., f(t-1)\}$, we can also use the RNN to generate $\{f(t), f(t+1), ..., f(T)\}$ to complete the sequence.

In the training stage, we are given the sequence $\{f(1), f(2), ..., f(T)\}$. During training, we keep Step 1 and 2 above. Since we have the ground-truth for $f(t+1)$ during training, we will compute the squared error $L_t = (f(t+1) - \mathbf{y}_t)^2$ for all $t \in \{1, 2, ..., T-1\}$ and optimize the average loss $L = \frac{1}{T-1}\sum_{t=1}^{T-1} L_t$. Note that $f(t+1) - \mathbf{y}_t$ is actually a scalar operation since $\mathbf{y}_t \in \mathbb{R}^1$ can be regarded as a scalar.

In PyTorch implementation, you only need to use `nn.RNNCell()` to perform $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1})$ and `nn.Linear()` to perform $\mathbf{y}_t = W_{hy}\mathbf{h}_t$. In the implementation, the input $\mathbf{x}_t$ and output $\mathbf{y}_t$ are both 1-dim vectors. The hidden state $\mathbf{h}_t \in \mathbb{R}^{100}$, which means there are 100 hidden neurons (i.e. hidden size = 100).

Note:

1. In the description above, we ignore the bias terms, while in PyTorch implementation, both `nn.RNNCell()` and `nn.Linear()` have bias terms at default. You can simply use the default setting of `nn.RNNCell()` and `nn.Linear()`.

Download the skeleton code `hw5-periodic.ipynb` from the course website. Complete the skeleton code. In order to get full marks, your report should include:

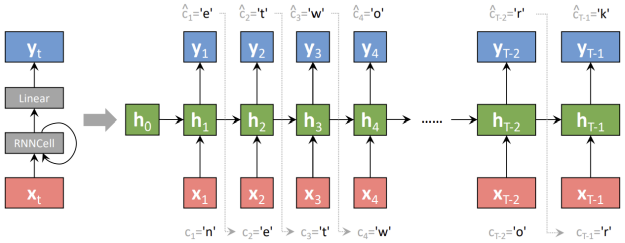
1. Your code.
2. Training loss curve.
3. A figure of generated sequence.

Hint: You may find the official PyTorch tutorial helpful before implementing the network.

Solution: Code and figure are attached at the end.

Problem 6. (Extra Credit - 15 points) Char-RNN

In section 2, you already obtain some experience with the forward procedure of RNN. In this section, you will train a special RNN called Char-RNN (i.e. Character RNN) on a language modeling task. Assume we have a character sequence $\{c_t\} = \{c_1, c_2..., c_{T-1}, c_T\}$ where $c_t \in A$ and A is the set of an alphabet (e.g. the character sequence can be $\{'n', 'e', 't', 'w', 'o', 'r', 'k'\}$, which will be used in the illustration below). We aim to train a network to predict the next character c_{t+1} given current character c_t. Thus, after training, if we have initial character c_1, we can iteratively generate the whole sequence $\{c_t\}$:



The right part of the figure shows the "unrolled" version of the Char-RNN. Assume $|A| = K$, then we can represent the characters in alphabet A into K classes $\{0, 1..., K-2, K-1\}$. Then, we have the following steps for sequence generation:

1. We convert the input character c_t into the one-hot encoding $\mathbf{x}_t \in \mathbb{R}^K$ of the class of the input character.
2. Based on previous hidden state $\mathbf{h}_{t-1}$ and one-hot input $\mathbf{x}_t$, we can calculate the current hidden state $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1})$ and predict the output: $\mathbf{y}_t = W_{hy}\mathbf{h}_t$.
3. Afterwards, we can use the softmax function to obtain the probability vector $\mathbf{p}_t = \text{Softmax}(\mathbf{y}_t) = \{p_0, ..., p_{K-1}\} \in \mathbb{R}^K$. Then, we will sample the class $\hat{k}$ from the probability vector $\mathbf{p}$ and find the corresponding character $\hat{c}_t$ as the predicted character. Note that here we do not use $\hat{k} = \arg\max_k p_k$ (i.e. the index with maximum probability) as prediction since it will reduce the diversity of the generated sequence.
4. We will use the current predicted character $\hat{c}_t$ as next input character c_{t+1} and perform the above steps again.

In the evaluation stage, if we have initial hidden state $\mathbf{h}_0$, initial input character c_1 and trained weight W_{xh}, W_{hh}, W_{hy}, we can utilize the above steps to generate the whole sequence $\{c_1, c_2, ..., c_T\}$.

In the training stage, we are given the sequence $\{c_1, c_2, ..., c_T\}$. During training, we keep Step 1 and 2 above and calculate the probability vector $\mathbf{p}_t$ in Step 3. Since we have the ground-truth for c_{t+1} during training, we will compute the cross-entropy loss $L_t = \text{CrossEntropy}(\mathbf{p}_t, c_{t+1})$ for all $t \in \{1, 2, ..., T\}$ and optimize the average loss $L = \frac{1}{T}\sum_{t=1}^{T} L_t$.

The left part of the figure shows the actual network in PyTorch implementation. You only need to use `nn.RNNCell()` to perform $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1})$ and `nn.Linear()` to perform $\mathbf{y}_t = W_{hy}\mathbf{h}_t$. In the implementation, we have the size of alphabet $K = 100$, which means one-hot input $\mathbf{x}_t$ and output $\mathbf{y}_t$ are both 100-dim vectors. The hidden state $\mathbf{h}_t \in \mathbb{R}^{100}$, which means there are 100 hidden neurons (i.e. hidden size = 100).

Note:

1. In the description above, we ignore the bias terms, while in PyTorch implementation, both `nn.RNNCell()` and `nn.Linear()` have bias terms at default. You can simply use the default setting of `nn.RNNCell()` and `nn.Linear()`.

Download the skeleton code `hw5-rnn.ipynb` from the course website. Complete the skeleton code. In order to get full marks, your report should include

1. Your code
2. Training loss curve
3. A sample of generated sequence

Hint: You may find the official PyTorch tutorial helpful before implementing the network.

Solution: Code and figure are attached at the end.

Problem 3

In [1]:

```
import numpy as np
```

Training (Constructing Weights)

In [8]:

```
states = np.array([[+1, -1, +1, -1, +1], [-1, +1, +1, -1, +1]])

# Number of states
N = len(states[0])
M = len(states)
# Construct the weight matrix
W = 1 / N * states.T @ states
for i in range(N):
    W[i][i] = 0
W
```

Out[8]:

```
array([[ 0. , -0.4,  0. ,  0. ,  0. ],
       [-0.4,  0. ,  0. ,  0. ,  0. ],
       [ 0. ,  0. ,  0. , -0.4,  0.4],
       [ 0. ,  0. , -0.4,  0. , -0.4],
       [ 0. ,  0. ,  0.4, -0.4,  0. ]])
```

Probing Pattern

In [3]:

```
initial_input_state = np.array([+1, +1, +1, +1, +1])
```

Dynamic Evolution

In [19]:

```
# Hard-limiting non-linearity function
f_h = np.vectorize(lambda x: 1 if x >= 0 else -1)

# Dynamic evolution
def evolve(order, log=False):
    t = 0
    last_U = None
    U = initial_input_state.copy()
    if log:
        print('U({}) = {}'.format(t, U))

    # Until convergence
    while not np.array_equal(U, last_U):
        t += 1
        last_U = U.copy()
        # Update the nodes in order
        for i in order:
            # Be sure to zero-index

            # YOUR CODE HERE
            U[i-1] = f_h(sum(W[j][i-1] * U[j] for j in range(N) if j != i-1))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            if log:
                print('U_{}({}) = {}'.format(i, t, U[i-1]))

        # Log U for this iteration
        if log:
            print('U({}) = {}'.format(t, U))
            None

    return U
```

In [21]:

```
U = evolve([3, 1, 5, 2, 4], log=True)
print(U)
```

```
U(0) = [1 1 1 1 1]
U_3(1) = 1
U_1(1) = -1
U_5(1) = 1
U_2(1) = 1
U_4(1) = -1
U(1) = [-1  1  1 -1  1]
U_3(2) = 1
U_1(2) = -1
U_5(2) = 1
U_2(2) = 1
U_4(2) = -1
U(2) = [-1  1  1 -1  1]
[-1  1  1 -1  1]
```

In [22]:

```
U = evolve([2, 4, 3, 5, 1], log=True)
print(U)
```

```
U(0) = [1 1 1 1 1]
U_2(1) = -1
U_4(1) = -1
U_3(1) = 1
U_5(1) = 1
U_1(1) = 1
U(1) = [ 1 -1  1 -1  1]
U_2(2) = -1
U_4(2) = -1
U_3(2) = 1
U_5(2) = 1
U_1(2) = 1
U(2) = [ 1 -1  1 -1  1]
[ 1 -1  1 -1  1]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

🧠 (Problem 4) Traveling Salesman Problem — Hopfield Neural Network

Interactive Classroom Demo

Run the final cell and watch the network solve the TSP live — no widgets needed.

Panel	What it shows
🗺️ **Route Map**	The inferred tour evolving on a geographic map
🔥 **V Matrix**	Neuron activations sharpening toward a valid permutation
⚡ **Energy Curve**	Network energy dropping as it converges

In [20]:

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.colors import LinearSegmentedColormap
from IPython.display import clear_output
import time
import warnings
warnings.filterwarnings('ignore')

print('Libraries loaded ✓')
```

```
Libraries loaded ✓
```

City Setup & Distance Matrix

In [21]:

```
cities = ['San Diego', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle']
N = len(cities)

coords = np.array([
    [-117.2, 32.7],
    [-118.2, 34.1],
    [ -87.6, 41.9],
    [ -74.0, 40.7],
    [ -71.1, 42.4],
    [-122.3, 47.6],
])

D_raw = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        D_raw[i, j] = np.sqrt(np.sum((coords[i] - coords[j])**2))
D_norm = D_raw / D_raw.max()

CITY_COLORS = [
    '#FF6B6B', # San Diego
    '#4ECDC4', # Los Angeles
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    '#45B7D1', # Chicago
    '#96CEB4', # New York
    '#FFEAA7', # Boston
    '#FF6B6B', # Seattle
]

BG     = '#1a1a2e'
PANEL  = '#16213e'
LBLUE  = '#4ECDC4'
GREEN  = '#0ead69'
YELLOW = '#f5a623'
ACCENT = '#e94560'

VMAP = LinearSegmentedColormap.from_list(
    'vmap', ['#1a1a2e', '#e94560', '#f5a623', '#ffffff'], N=256
)

print('Cities configured ✓')
```

```
Cities configured ✓
```

Hopfield Network

In [22]:

```
class HopfieldTSP:
    def __init__(self, N, D, A=500, B=500, C=200, D_coeff=100, dt=1e-4, gain=50):
        self.N, self.D = N, D
        self.A, self.B, self.C, self.D_coeff = A, B, C, D_coeff
        self.dt, self.gain = dt, gain
        self.U = 0.02 * np.random.randn(N, N)
        self.V = self._sig(self.U)

    def _sig(self, u):
        return 0.5 * (1 + np.tanh(u * self.gain / 2))

    def step(self):
        V, N = self.V, self.N
        dU = np.zeros((N, N))
        total = np.sum(V)
        for i in range(N):
            for j in range(N):
                jn, jp = (j+1)%N, (j-1)%N
                dU[i,j] = (
                    - self.A * (np.sum(V[i,:]) - V[i,j])
                    - self.B * (np.sum(V[:,j]) - V[i,j])
                    - self.C * (total - N)
                    - self.D_coeff * np.dot(self.D[i,:], V[:,jn] + V[:,jp])
                )
        self.U += self.dt * (-self.U + dU)
        self.V = self._sig(self.U)

    def energy(self):
        V, N = self.V, self.N
        E  = self.A/2 * sum((np.sum(V[i,:])**2 - np.sum(V[i,:]**2)) for i in range(N))
        E += self.B/2 * sum((np.sum(V[:,j])**2 - np.sum(V[:,j]**2)) for j in range(N))
        E += self.C/2 * (np.sum(V) - N)**2
        d  = sum(self.D[i,:] @ V[:,(j+1)%N] * V[i,j] for i in range(N) for j in range(N))
        E += self.D_coeff/2 * d
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        return E

    def decode_tour(self):
        t = np.argmax(self.V, axis=0).tolist()
        return t if len(set(t)) == self.N else None

print('HopfieldTSP defined ✓')
```

```
HopfieldTSP defined ✓
```

▶ Run This Cell to Watch the Network Converge Live

In [29]:

```
# ── Settings ──────────────────────────────────────────────────────
N_STEPS      = 3000   # total network iterations
DISPLAY_EVERY = 150   # show a frame every N steps  ← adjust for speed
FRAME_DELAY  = 0.1    # seconds between frames      ← adjust for speed

# ── Find a seed that produces a valid tour ────────────────────────
# print('Searching for a good random seed...')
# good_seed = 0
# for seed in np.random.randint(0, 1000000, 100):
#     np.random.seed(seed)
#     test = HopfieldTSP(N, D_norm)
#     for _ in range(N_STEPS):
#         test.step()
#     if test.decode_tour() is not None:
#         good_seed = seed
#         print(f'Seed {seed} gives a valid tour ✓')
#         break

# ── Main run with live plotting ───────────────────────────────────
good_seed = 80163
np.random.seed(good_seed)
net = HopfieldTSP(N, D_norm)

energy_history = []
step_history   = []
frame_num      = 0

display_steps = list(range(1, DISPLAY_EVERY, 10))          # a few early frames
display_steps += list(range(DISPLAY_EVERY, N_STEPS+1, DISPLAY_EVERY))
display_steps = sorted(set(display_steps))

for s in range(1, N_STEPS + 1):
    net.step()
    e = net.energy()
    energy_history.append(e)
    step_history.append(s)

    if s not in display_steps:
        continue

    frame_num += 1
    tour  = net.decode_tour()
    V     = net.V
    valid = tour is not None
    status_color = GREEN  if valid else YELLOW
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    status_text  = '✓  VALID TOUR FOUND' if valid else '⟳  Converging…'
    sharpness = (np.mean(np.max(V, axis=1)) + np.mean(np.max(V, axis=0))) / 2

    # ── Build figure ──────────────────────────────────────────────
    fig = plt.figure(figsize=(17, 9), facecolor=BG)
    gs  = gridspec.GridSpec(
        2, 3, figure=fig,
        left=0.05, right=0.97, top=0.88, bottom=0.09,
        wspace=0.38, hspace=0.52
    )

    # Title
    fig.text(0.5, 0.95,
             '🧠  Hopfield Neural Network — TSP Convergence',
             ha='center', fontsize=16, fontweight='bold', color='white')
    fig.text(0.5, 0.906,
             f'Step {s:>4d} / {N_STEPS}          {status_text}',
             ha='center', fontsize=12, color=status_color, fontweight='bold')

    # ── Panel A: Route Map ────────────────────────────────────────
    ax_map = fig.add_subplot(gs[0, :2])
    ax_map.set_facecolor(PANEL)
    for sp in ax_map.spines.values(): sp.set_edgecolor('#333366')

    x, y = coords[:,0], coords[:,1]

    if valid:
        for i in range(N):
            c1, c2 = tour[i], tour[(i+1)%N]
            ax_map.annotate('',
                xy=(x[c2], y[c2]), xytext=(x[c1], y[c1]),
                arrowprops=dict(arrowstyle='->', color=LBLUE, lw=2.5,
                                connectionstyle='arc3,rad=0.12', mutation_scale=20))
    else:
        for i in range(N):
            for j in range(N):
                for k in range(N):
                    w = V[i,j] * V[k,(j+1)%N]
                    if w > 0.06:
                        ax_map.plot([x[i], x[k]], [y[i], y[k]],
                                    color=LBLUE, alpha=min(w*1.5, 0.5), lw=1.5)

    for i in range(N):
        ax_map.scatter(x[i], y[i], s=300, color=CITY_COLORS[i],
                       zorder=6, edgecolors='white', linewidths=1.8)
        ax_map.text(x[i], y[i]+0.85, cities[i],
                    ha='center', fontsize=10, fontweight='bold', color='white', zorder=7)
        if valid and i in tour:
            ax_map.text(x[i], y[i]-1.1, f'#{tour.index(i)+1}',
                        ha='center', fontsize=9, color=CITY_COLORS[i],
                        fontweight='bold', zorder=7)

    ax_map.set_xlim(-124, -62)
    ax_map.set_ylim(30, 50)
    ax_map.set_title('🗺  Route Map', fontsize=12, fontweight='bold', color='white', pad
    ax_map.set_xlabel('Longitude', fontsize=9, color='#aaaacc')
    ax_map.set_ylabel('Latitude',  fontsize=9, color='#aaaacc')
    ax_map.grid(True, alpha=0.12, color='white')
    ax_map.tick_params(colors='#aaaacc', labelsize=8)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    if valid:
        seq = '  →  '.join(cities[c] for c in tour) + f'  →  {cities[tour[0]]}'
        ax_map.text(0.5, -0.15, seq, transform=ax_map.transAxes,
                    ha='center', fontsize=9, color=LBLUE, fontweight='bold')
    else:
        ax_map.text(0.5, -0.15, 'Route undetermined — neurons still competing',
                    transform=ax_map.transAxes,
                    ha='center', fontsize=9, color=YELLOW, style='italic')

    # ── Panel B: V Matrix ──────────────────────────────────────────
    ax_v = fig.add_subplot(gs[0, 2])
    ax_v.set_facecolor(PANEL)

    im = ax_v.imshow(V, cmap=VMAP, vmin=0, vmax=1, aspect='auto')
    for i in range(N):
        for j in range(N):
            val = V[i, j]
            ax_v.text(j, i, f'{val:.2f}',
                      ha='center', va='center', fontsize=9,
                      color='white' if val < 0.6 else '#1a1a2e',
                      fontweight='bold' if val > 0.7 else 'normal')

    ax_v.set_xticks(range(N))
    ax_v.set_xticklabels([f'Pos {k+1}' for k in range(N)], fontsize=8, color='#aaaacc')
    ax_v.set_yticks(range(N))
    ax_v.set_yticklabels([c[:9] for c in cities], fontsize=8, color='#aaaacc')
    ax_v.set_title('🔥  V Matrix  [city × position]',
                   fontsize=10, fontweight='bold', color='white', pad=7)
    cb = plt.colorbar(im, ax=ax_v, fraction=0.046, pad=0.04)
    cb.ax.tick_params(colors='#aaaacc', labelsize=7)
    cb.set_label('Activation', color='#aaaacc', fontsize=7)

    ax_v.text(0.5, -0.13,
              f'Sharpness: {sharpness:.3f}  (1.0 = perfect assignment)',
              transform=ax_v.transAxes, ha='center', fontsize=8,
              color=GREEN if sharpness > 0.9 else YELLOW)

    # ── Panel C: Energy Curve ──────────────────────────────────────
    ax_e = fig.add_subplot(gs[1, :])
    ax_e.set_facecolor(PANEL)
    for sp in ax_e.spines.values(): sp.set_edgecolor('#333366')

    ax_e.plot(step_history, energy_history, color=LBLUE, lw=2)
    ax_e.fill_between(step_history, energy_history, alpha=0.08, color=LBLUE)
    ax_e.scatter([s], [e], s=140, color=status_color,
                 zorder=6, edgecolors='white', linewidths=1.5)
    ax_e.annotate(f'  E = {e:.1f}', (s, e),
                  fontsize=9, color=status_color, fontweight='bold', va='center')

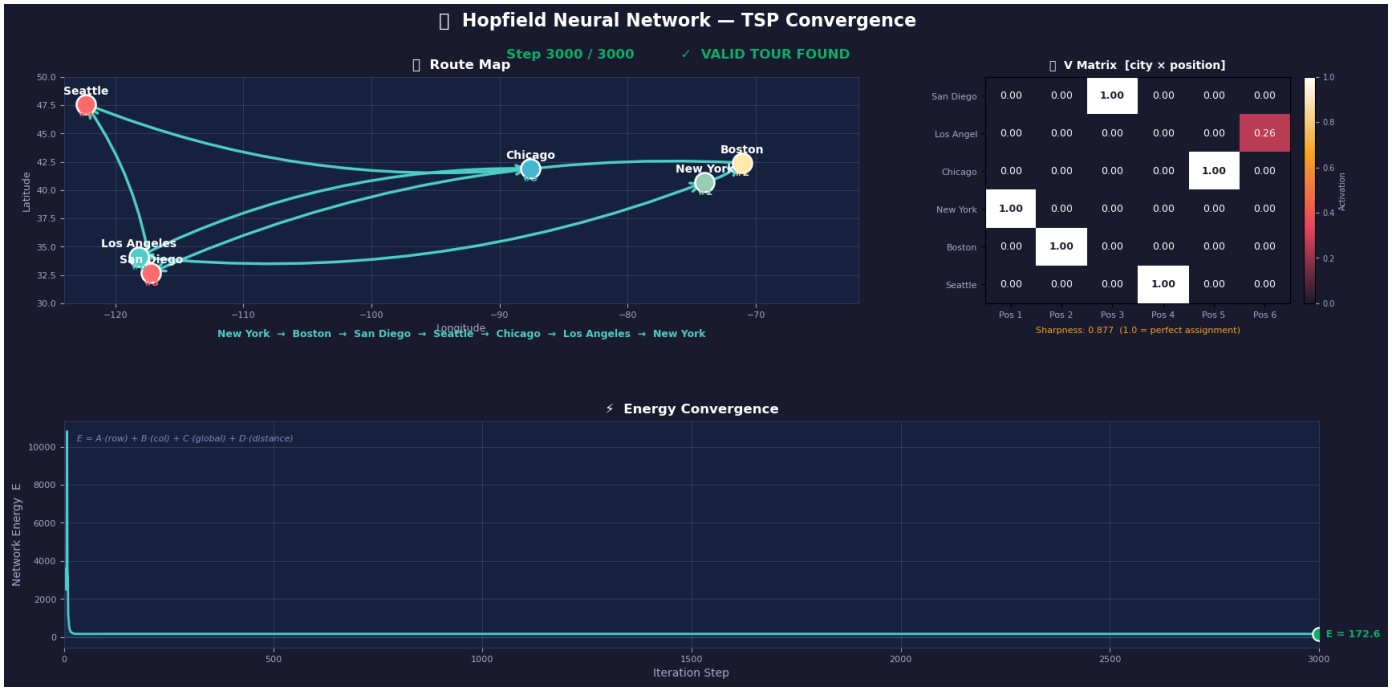
    ax_e.set_xlim(0, N_STEPS)
    ax_e.set_xlabel('Iteration Step', fontsize=10, color='#aaaacc')
    ax_e.set_ylabel('Network Energy  E', fontsize=10, color='#aaaacc')
    ax_e.set_title('⚡  Energy Convergence',
                   fontsize=12, fontweight='bold', color='white', pad=7)
    ax_e.grid(True, alpha=0.12, color='white')
    ax_e.tick_params(colors='#aaaacc', labelsize=8)
    ax_e.text(0.01, 0.91,
              'E = A·(row) + B·(col) + C·(global) + D·(distance)',
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            transform=ax_e.transAxes, fontsize=8,
            color='#8888bb', style='italic')

    clear_output(wait=True)
    plt.show()
    time.sleep(FRAME_DELAY)

print('\n✅ Done! Scroll up to review all convergence frames.')
```




```
✅ Done! Scroll up to review all convergence frames.
```

In [30]:

```
net.decode_tour()
```

Out[30]:

```
[3, 4, 0, 5, 2, 1]
```

📖 Teaching Notes

Stage	V Matrix	Route Map	Energy
Early	All cells ≈ equal — neurons uncertain	Soft overlapping connections	High, noisy
Mid	Some cells brightening — competition starting	Structure emerging, may have conflicts	Dropping steeply
Late	One bright cell per row & column — valid permutation	Single clean tour with stop numbers	Plateau reached

Adjust `DISPLAY_EVERY` and `FRAME_DELAY` in the run cell to control the speed of the demo.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 5

In [1]:

```
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
```

In [2]:

```
import string
import random
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
```

Prepare for Dataset

In [3]:

```
# Get a random sequence of sine curve.
def get_random_seq():
    seq_len     = 128  # The length of an input sequence.
    # Sample a sequence.
    t = np.arange(0, seq_len)
    a = 2*np.pi*1.0/seq_len
    b = 2*np.pi*np.random.rand()*5
    seq = np.sin(a*t+b)
    return seq

# Sample a mini-batch including input tensor and target tensor.
def get_input_and_target():
    seq    = get_random_seq()
    input  = torch.tensor(seq[:-1]).float().view(-1,1,1) # Input sequence.
    target = torch.tensor(seq[1:]).float().view(-1,1,1)  # Target sequence.
    return input, target
```

Choose a Device

In [4]:

```
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cpu
```

Network Definition

In [5]:

```
class Net(nn.Module):
    def __init__(self):
        # Initialization.
        super(Net, self).__init__()
        self.input_size  = 1
        self.hidden_size = 100
        self.output_size = 1
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        self.rnn_cell = nn.RNNCell(self.input_size, self.hidden_size)
        self.linear   = nn.Linear(self.hidden_size, self.output_size)

    def forward(self, input, hidden):
        """ Forward function.
              input:  Input. It refers to the x_t in homework write-up.
              hidden: Previous hidden state. It refers to the h_{t-1}.
            Returns (output, hidden) where output refers to y_t and
                     hidden refers to h_t.
        """
        # Forward function.
        h_t = self.rnn_cell(input, hidden)
        y_t = self.linear(h_t)
        return y_t, h_t

    def init_hidden(self):
        # Initial hidden state.
        # 1 means batch size = 1.
        return torch.zeros(1, self.hidden_size).to(device)

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[5]:

```
Net(
  (rnn_cell): RNNCell(1, 100)
  (linear): Linear(in_features=100, out_features=1, bias=True)
)
```

Training Step and Evaluation Step

In [6]:

```
loss_func = lambda y, t: (y - t)**2

# Training step function.
def train_step(net, opt, input, target):
    """ Training step.
        net:    The network instance.
        opt:    The optimizer instance.
        input:  Input tensor.  Shape: [seq_len, 1, 1].
        target: Target tensor. Shape: [seq_len, 1].
    """
    seq_len = input.shape[0]    # Get the sequence length of current input.
    hidden = net.init_hidden()  # Initial hidden state.
    net.zero_grad()             # Clear the gradient.
    loss = 0                    # Initial loss.

    for t in range(seq_len):    # For each one in the input sequence.
        output, hidden = net(input[t], hidden)
        loss += loss_func(output, target[t])

    loss.backward()             # Backward.
    opt.step()                  # Update the weights.

    return loss / seq_len       # Return the average loss w.r.t sequence length.
```

In [7]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Evaluation step function.
def eval_step(net, predicted_len=100):
    # Initialize the hidden state, input and the predicted sequence.
    hidden        = net.init_hidden()
    init_seq      = get_random_seq()
    init_input    = torch.tensor(init_seq).float().view(-1,1,1).to(device)
    predicted_seq = []

    # Use initial points on the curve to "build up" hidden state.
    for t in range(len(init_seq) - 1):
        output, hidden = net(init_input[t], hidden)

    # Set current input as the last character of the initial string.
    input = init_input[-1]

    # Predict more points after the initial string.
    for t in range(predicted_len):
        # Get the current output and hidden state.
        output, hidden = net(input, hidden)

        # Add predicted point to the sequence and use it as next input.
        predicted_seq.append(output.item())

        # Use the predicted point to generate the input of next round.
        input = output

    return init_seq, predicted_seq
```

Training Procedure

In [8]:

```
# Number of iterations.
iters       = 200    # Number of training iterations.
print_iters = 10     # Number of iterations for each log printing.

# The loss variables.
all_losses = []
loss_sum   = 0

# Initialize the optimizer and the loss function.
opt       = torch.optim.Adam(net.parameters(), lr=0.005)
loss_func = nn.MSELoss()

# Training procedure.
for i in range(iters):
    input, target = get_input_and_target()            # Fetch input and target.
    input, target = input.to(device), target.to(device) # Move to GPU memory.
    loss      = train_step(net, opt, input, target)   # Calculate the loss.
    loss_sum += loss                                  # Accumulate the loss.

    # Print the log.
    if i % print_iters == print_iters - 1:
        print('iter:{}/{} loss:{}'.format(i, iters, loss_sum / print_iters))
        #print('generated sequence: {}\n'.format(eval_step(net)))

        # Track the loss.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

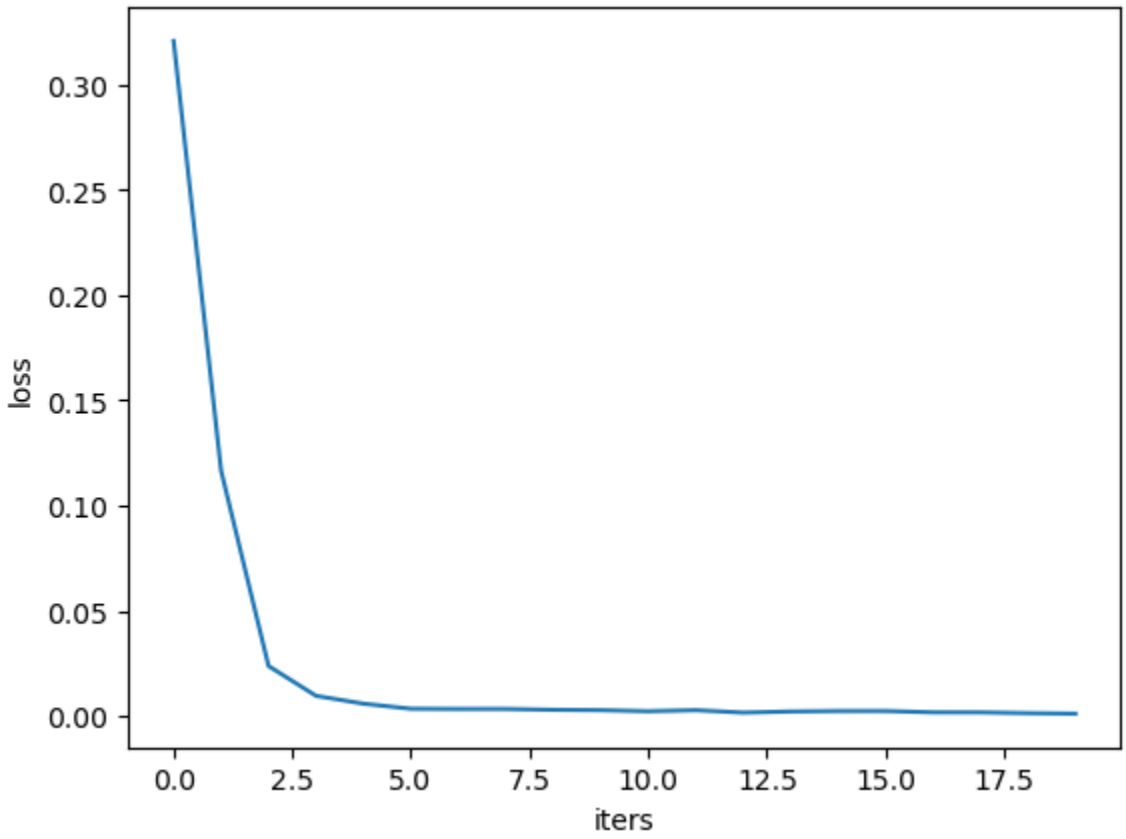
```
        all_losses.append((loss_sum / print_iters).item())
        loss_sum = 0
```

```
iter:9/200 loss:0.32080748677253723
iter:19/200 loss:0.11659383773803711
iter:29/200 loss:0.023761939257383347
iter:39/200 loss:0.009609444066882133
iter:49/200 loss:0.005872826091945171
iter:59/200 loss:0.003459220752120018
iter:69/200 loss:0.0033439001999795437
iter:79/200 loss:0.003350761253386736
iter:89/200 loss:0.0029689432121813297
iter:99/200 loss:0.002794354921206832
iter:109/200 loss:0.002278412925079465
iter:119/200 loss:0.002791750244796276
iter:129/200 loss:0.0016406966606155038
iter:139/200 loss:0.0021623976062983274
iter:149/200 loss:0.002376296790316701
iter:159/200 loss:0.002387423301115632
iter:169/200 loss:0.0017720026662573218
iter:179/200 loss:0.001747651258483529
iter:189/200 loss:0.0013632839545607567
iter:199/200 loss:0.001131489989347756
```

Training Loss Curve

In [9]:

```
plt.xlabel('iters')
plt.ylabel('loss')
plt.plot(all_losses)
plt.show()
```

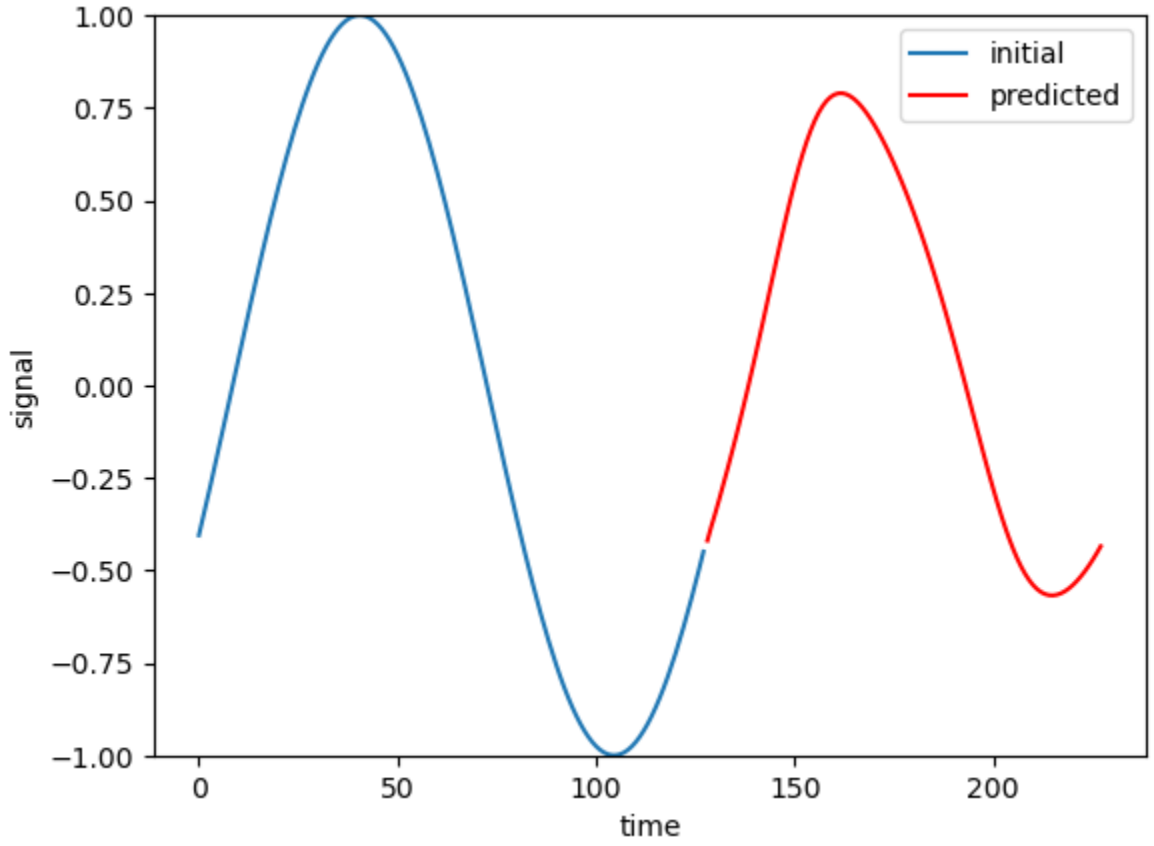


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Evaluation: A Sample of Generated Sequence

In [10]:

```
init_seq, predicted_seq = eval_step(net, predicted_len=100)
init_t      = np.arange(0, len(init_seq))
predicted_t = np.arange(len(init_seq), len(init_seq)+len(predicted_seq))
plt.plot(init_t, init_seq, label='initial')
plt.plot(predicted_t, predicted_seq, color='red', label='predicted')
plt.legend()
plt.ylim([-1, 1])
plt.xlabel('time')
plt.ylabel('signal')
plt.show()
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 6

In [1]:
```
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
```

In [2]:
```
import string
import random
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

Prepare for Dataset

In [3]:
```
all_chars       = string.printable
n_chars         = len(all_chars)
file            = open('./shakespeare.txt').read()
file_len        = len(file)

print('Length of file: {}'.format(file_len))
print('All possible characters: {}'.format(all_chars))
print('Number of all possible characters: {}'.format(n_chars))
```

```
Length of file: 1115394
All possible characters: 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVW
XYZ!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~

Number of all possible characters: 100
```

In [4]:
```
# Get a random sequence of the Shakespeare dataset.
def get_random_seq():
    seq_len     = 128  # The length of an input sequence.
    start_index = random.randint(0, file_len - seq_len)
    end_index   = start_index + seq_len + 1
    return file[start_index:end_index]

# Convert the sequence to one-hot tensor.
def seq_to_onehot(seq):
    tensor = torch.zeros(len(seq), 1, n_chars)
    # Shape of the tensor:
    #     (sequence length, batch size, classes)
    # Here we use batch size = 1 and classes = number of unique characters.
    for t, char in enumerate(seq):
        index = all_chars.index(char)
        tensor[t][0][index] = 1
    return tensor

# Convert the sequence to index tensor.
def seq_to_index(seq):
    tensor = torch.zeros(len(seq), 1)
    # Shape of the tensor:
    #     (sequence length, batch size).
    # Here we use batch size = 1.
```

```
    for t, char in enumerate(seq):
        tensor[t] = all_chars.index(char)
    return tensor

# Sample a mini-batch including input tensor and target tensor.
def get_input_and_target():
    seq    = get_random_seq()
    input  = seq_to_onehot(seq[:-1])      # Input is represented in one-hot.
    target = seq_to_index(seq[1:]).long() # Target is represented in index.
    return input, target
```

Choose a Device

In [5]:
```
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cpu
```

Network Definition

In [6]:
```
class Net(nn.Module):
    def __init__(self):
        # Initialization.
        super(Net, self).__init__()
        self.input_size  = n_chars   # Input size: Number of unique chars.
        self.hidden_size = 100       # Hidden size: 100.
        self.output_size = n_chars   # Output size: Number of unique chars.

        self.rnn_cell = nn.RNNCell(self.input_size, self.hidden_size)
        self.linear   = nn.Linear(self.hidden_size, self.output_size)

    def forward(self, input, hidden):
        """ Forward function.
              input:  One-hot input. It refers to the x_t in homework write-up.
              hidden: Previous hidden state. It refers to the h_{t-1}.
            Returns (output, hidden) where output refers to y_t and
                     hidden refers to h_t.
        """
        # Forward function.
        hidden = self.rnn_cell(input, hidden)
        output = self.linear(hidden)

        return output, hidden

    def init_hidden(self):
        # Initial hidden state.
        # 1 means batch size = 1.
        return torch.zeros(1, self.hidden_size).to(device)

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[6]:
```
Net(
  (rnn_cell): RNNCell(100, 100)
  (linear): Linear(in_features=100, out_features=100, bias=True)
)
```

Training Step and Evaluation Step

In [7]:
```python
loss_func = nn.CrossEntropyLoss()

# Training step function.
def train_step(net, opt, input, target):
    """ Training step.
        net:    The network instance.
        opt:    The optimizer instance.
        input:  Input tensor.  Shape: [seq_len, 1, n_chars].
        target: Target tensor. Shape: [seq_len, 1].
    """
    seq_len = input.shape[0]    # Get the sequence length of current input.
    hidden = net.init_hidden()  # Initial hidden state.
    net.zero_grad()             # Clear the gradient.
    loss = 0                    # Initial loss.

    for t in range(seq_len):    # For each one in the input sequence.
        output, hidden = net(input[t], hidden)
        loss += loss_func(output, target[t])

    loss.backward()             # Backward.
    opt.step()                  # Update the weights.

    return loss / seq_len       # Return the average loss w.r.t sequence length.
```

In [8]:
```python
# Evaluation step function.
def eval_step(net, init_seq='W', predicted_len=100):
    # Initialize the hidden state, input and the predicted sequence.
    hidden        = net.init_hidden()
    init_input    = seq_to_onehot(init_seq).to(device)
    predicted_seq = init_seq

    # Use initial string to "build up" hidden state.
    for t in range(len(init_seq) - 1):
        output, hidden = net(init_input[t], hidden)

    # Set current input as the last character of the initial string.
    input = init_input[-1]

    # Predict more characters after the initial string.
    for t in range(predicted_len):
        # Get the current output and hidden state.
        output, hidden = net(input, hidden)

        # Sample from the output as a multinomial distribution.
        predicted_index = torch.multinomial(output.view(-1).exp(), 1)[0]

        # Add predicted character to the sequence and use it as next input.
        predicted_char  = all_chars[predicted_index]
```

```
        predicted_seq  += predicted_char

        # Use the predicted character to generate the input of next round.
        input = seq_to_onehot(predicted_char)[0].to(device)

    return predicted_seq
```

Training Procedure

In [9]:
```
# Number of iterations.
iters       = 20000  # Number of training iterations.
print_iters = 100    # Number of iterations for each log printing.

# The loss variables.
all_losses = []
loss_sum   = 0

# Initialize the optimizer and the loss function.
opt       = torch.optim.Adam(net.parameters(), lr=0.005)
loss_func = nn.CrossEntropyLoss()

# Training procedure.
for i in range(iters):
    input, target = get_input_and_target()            # Fetch input and target.
    input, target = input.to(device), target.to(device) # Move to GPU memory.
    loss      = train_step(net, opt, input, target)   # Calculate the loss.
    loss_sum += loss                                  # Accumulate the loss.

    # Print the log.
    if i % print_iters == print_iters - 1:
        print('iter:{}/{} loss:{}'.format(i, iters, loss_sum / print_iters))
        print('generated sequence: {}\n'.format(eval_step(net)))

        # Track the loss.
        all_losses.append((loss_sum / print_iters).item())
        loss_sum = 0
```

```
iter:19799/20000 loss:1.7809001207351685
generated sequence: WO
TENRELANE:
You shears,
Preasoled
To recronthing as his spake,
To see, the rides will tencest ire h

iter:19899/20000 loss:1.7936503887176514
generated sequence: Wit
To drat us.

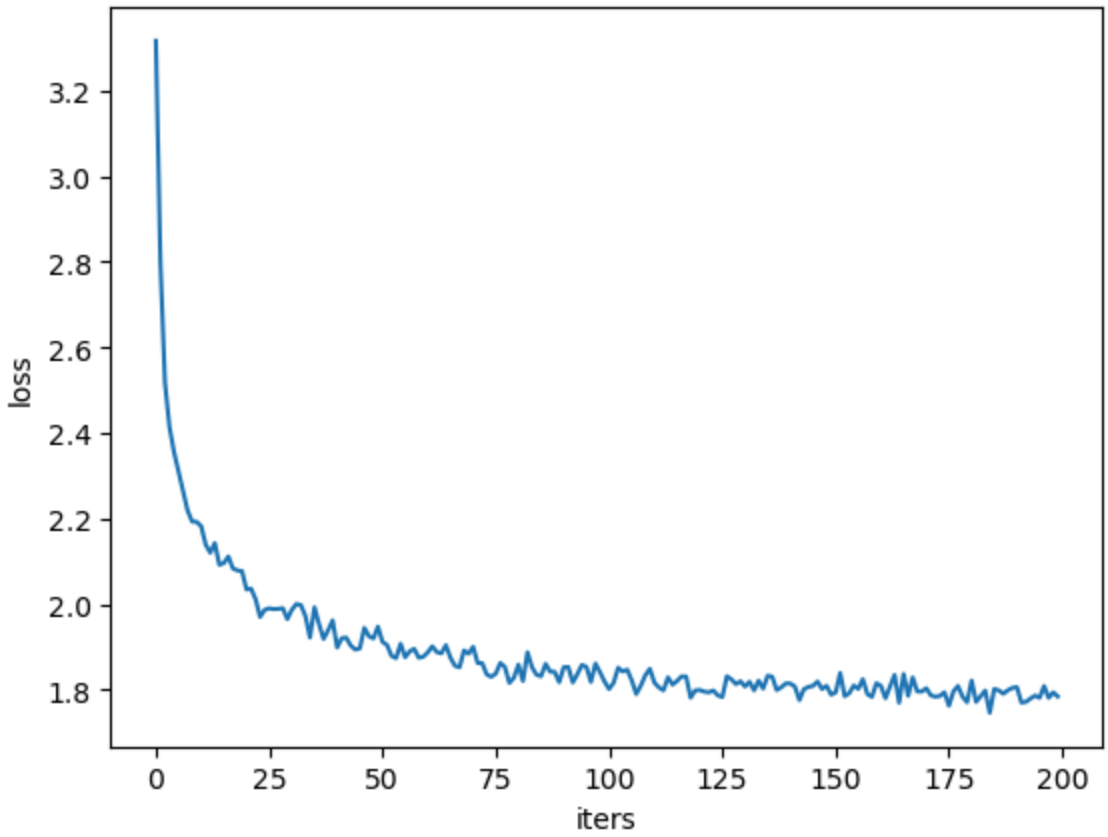
GLOUTES:
M' Kcem is 'Claider
Alm:s.
O, thou sir so LAUDE LORK:
They nothent's awhy,

iter:19999/20000 loss:1.7841639518737793
generated sequence: Why:
Most
rought out upon.

Lick aunt her wilt.
Ahe denverst, it may breads as the madien was
I jover
```

Training Loss Curve

In [10]:
```
plt.xlabel('iters')
plt.ylabel('loss')
plt.plot(all_losses)
plt.show()
```



Evaluation: A Sample of Generated Sequence

In [11]:
```
print(eval_step(net, predicted_len=600))
```

```
Whee,
Dook thinar: hath and groken
First,
In made but borss, the facery is;
Is this some EDthactry you malimanged with seevous; my
Fames; theh, bue be ote! good Andinurst to tringeinioblion:
my very poolly by thou nott.

LAUTER:
I you queer, my marraces for you, befice
Decouds but crow foo!
That stong by with it he'll oun nook, Buarh in liven'd of you my mest nome.

WARTIUMERD II:
Mean
Than hy've, and Enates an my pristess thriss to he's disfal'd.
For thy foow keet that you
Porry.

BUCKINGHAM: love my brolity ot hop youbpurding my liveng, solands:
Yen:
Fhall you, do you; his king
That he,
And b
```